

FINAL COURSE PROJECT

Selected Topics in Signal Processing

Semester: Spring 2026

Optimal and Adaptive Filtering Control

Faculty Of Engineering - Ain Shams University
ECE Department

SUBMITTED TO

Prof. Michael Ibrahim – Eng. Ahmed Khaled

Department of Signals and Communication Engineering

DATE OF SUBMISSION

June 16, 2026

Academic Session 2026

PREPARED BY

Student Name	Student ID
Ahmed Khairat Mohamed	2100157
Ahmed Alaa Mohamed	2101434

Contents

1 Task 1	2
1.1 Introduction	2
1.2 Methodology	2
1.3 Plots and Results	3
2 Task 2	4
2.1 Introduction	4
2.2 Methodology	4
2.3 Plots and Results	5

1 Task 1

1.1 Introduction

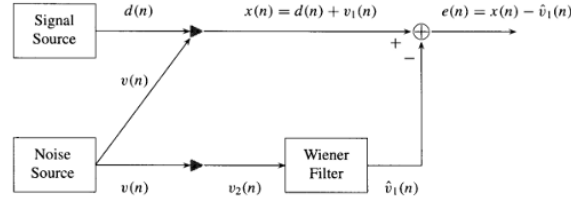


Figure 1: Filter Design

The main goal of this task is to design a Wiener filter that can clean up a noisy signal. We want to extract a clean sine wave, $d(n)$, from a blurry observation, $x(n)$. The main signal is messed up by one noise source (v_1), but our filter will use a second, related noise source (v_2) to help cancel out the unwanted interference. Both of these noise sources are modeled as simple AR(1) processes driven by basic white noise. The main equation of the desired function is $d(n) = \sin(0.05\pi n + \phi)$

1.2 Methodology

There is a category of equations called Wiener Hopf equations that plays a vital role in solving filter problems, but depending on the case we will study the Wiener Hopf equation will take different format according to depending which signal is the observed and which is the desired.

We calculate the best weights for the Wiener filter by solving the standard Wiener-Hopf equations. Instead of depending on impractical averages, we calculate the correlation matrix and vector using the actual data sequences we generated. Finally, we run the filter at two different orders, $M = 6$ and $M = 12$, to see how it performs.

The Wiener Hopf Equation that will be used in our case is

$$R_{v_2} w_{opt} = r_{xv_2}$$

1.3 Plots and Results

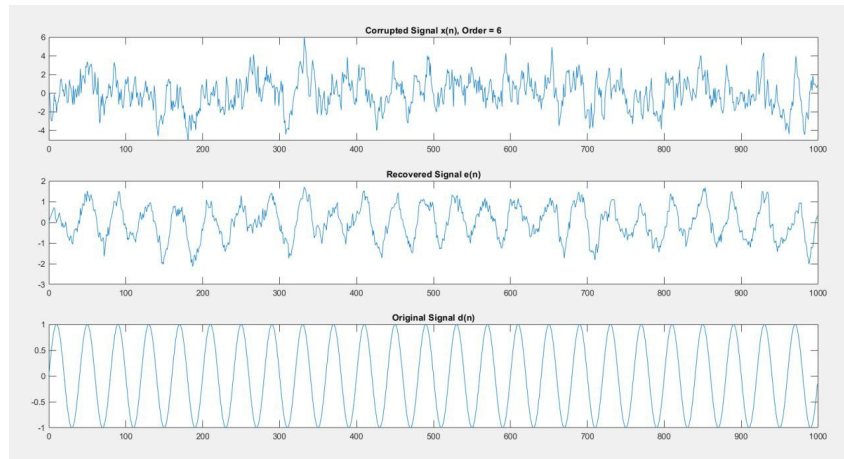


Figure 2: Plotting using filter of order 6.

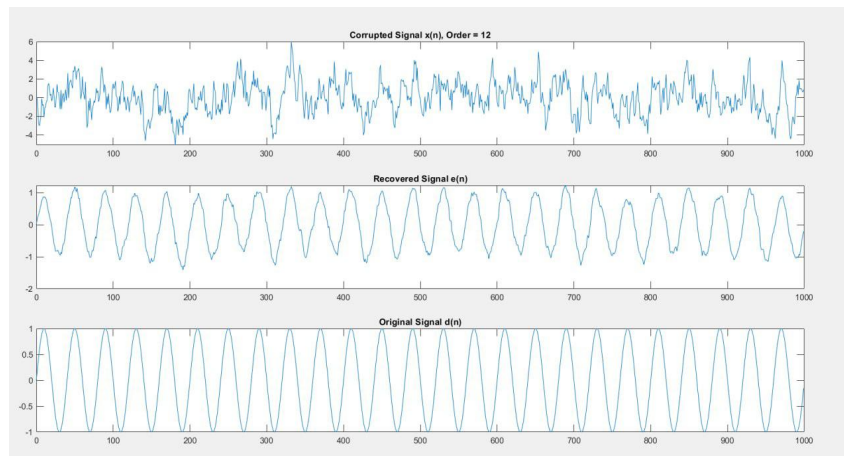


Figure 3: Plotting using filter of order 12.

Our plots prove that the noise cancellation works well to recover the clean sinusoidal wave. The $M = 12$ filter is clearly more accurate at tracing the desired signal than the lower-order $M = 6$ filter. While the $M = 6$ line still suffers from visible bumps and noise ripples, the $M = 12$ line provides a steady, accurate estimate, but the results show that the AR(1) background noise was successfully filtered out regardless using which order. Using higher order will lead to higher noise cancellation only.

2 Task 2

2.1 Introduction

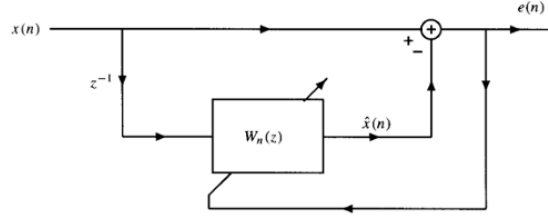


Figure 4: Filter Design

The main goal of this task is to design and implement an adaptive linear predictor using the Least Mean Squares (LMS) algorithm. We are working with a signal, $x(n)$, generated by a second-order autoregressive (AR(2)) process. The objective is to configure the LMS algorithm to automatically adjust the filter weights, $w_n(1)$ and $w_n(2)$, so that the prediction $\hat{x}(n)$ tracks the true signal as accurately as possible. By minimizing the prediction error, $e(n)$, over time, the system should adaptively converge toward the ideal theoretical parameters of the signal

2.2 Methodology

To establish the framework for our adaptive linear predictor, we must first analyze the statistical properties of the input signal. The sequence $x(n)$ is modeled as a second-order autoregressive (AR(2)) process driven by a zero-mean white noise source $v(n)$ with unit variance ($\sigma_v^2 = 1$):

$$x(n) = 1.2728x(n-1) - 0.81x(n-2) + v(n) \quad (1)$$

2.2.1 1. Autocorrelation Parameter Estimation

We can express the higher-order lag parameters as functions of the zero-lag variance $r_x(0)$:

$$\begin{aligned} r_x(1) &= 1.2728r_x(0) - 0.81r_x(1) \\ r_x(1) &= \frac{1.2728}{1 + 0.81}r_x(0) \approx 0.7032r_x(0) \end{aligned} \quad (2)$$

$$r_x(2) = 1.2728r_x(1) - 0.81r_x(0) \approx 0.0850r_x(0) \quad (3)$$

Incorporating the white noise variance contribution at the zero-lag boundary, the total signal power is defined by the following system balance:

$$r_x(0) = 1.2728r_x(1) - 0.81r_x(2) + 1 \quad (4)$$

Solving this simultaneous system of linear equations yields the exact statistical metrics for our process:

$$r_x(0) = 5.7523 \quad (5)$$

$$r_x(1) = 4.0451 \quad (6)$$

2.2.2 2. Correlation Matrix Eigenspace Bounds

The tracking capability and convergence speed of the LMS algorithm depend directly on the eigenvalues of the input correlation matrix R_x . For a second-order system, these matrix boundary limits are explicitly calculated using the primary autocorrelation coefficients:

$$\lambda_{\max} = r_x(0) + r_x(1) = 5.7523 + 4.0451 = 9.7974 \quad (7)$$

$$\lambda_{\min} = r_x(0) - r_x(1) = 5.7523 - 4.0451 = 1.7072 \quad (8)$$

2.2.3 3. LMS Algorithmic Stability Condition

To guarantee that the adaptive linear predictor converges and remains stable over time, the algorithm's step-size parameter μ must be strictly bounded by the inverse of the maximum eigenvalue:

$$0 < \mu < \frac{2}{\lambda_{\max}} \quad (9)$$

$$0 < \mu < 0.2041 \quad (10)$$

Although the μ max value is 0.2041 but at designing in real applications we will choose less than this value by one order so we will choose the μ max value to be 0.02041.

This is due to tough approximation made through our calculations as in LMS algorithm we assume the ergodicity so we calculate the ensemble mean from the average mean, then we assume that we are only working on one sample rather than L samples so we will take a margin of 0.1 as a design wise.

2.3 Plots and Results

```

===== SUMMARY =====
Optimum coefficients : w1 = 1.2728, w2 = -0.81
Minimum MSE          : xi_min = sigma_v^2 = 1
r_x(0) = 5.7523, r_x(1) = 4.0451
R = [5.7523  4.0451;
     4.0451  5.7523]
lambda_max = 9.7974, lambda_min = 1.7073
Step-size bound: 0 < mu < 0.2041

--- Autocorrelation (analytical via residues, causal poles only) ---
r_x(0) = 5.7523 (lecture answer: 5.7524)
r_x(1) = 4.0451 (lecture answer: 4.0451)
r_x(2) = 0.4892

--- Autocorrelation Matrix R ---
R = [5.7523  4.0451]
     [4.0451  5.7523]
(lecture answer: R = [5.7524  4.0451; 4.0451  5.7524])

--- Eigenvalues of R ---
lambda_max = 9.7974 (lecture answer: 9.7975)
lambda_min = 1.7073

--- Step-size Upper Bound: 0 < mu < 2/lambda_max ---
mu_max = 2 / 9.7974 = 0.2041 (lecture answer: 0.2041)

Note: mu should be ~1 order of magnitude below mu_max.
mu=0.02 is a good choice (lecture recommendation).
Both 0.02 and 0.04 satisfy the bound.

--- Partial Fraction Expansion (residues) ---
Poles:
0.7857 + 0.7857i
0.7857 - 0.7857i
0.6364 + 0.6364i
0.6364 - 0.6364i

Residues:
-2.8762 - 0.3019i
-2.8762 + 0.3019i
2.8762 - 0.3019i
2.8762 + 0.3019i

Poles:
1.1111
1.1111
0.9000
0.9000

--- Causal poles (|p| < 1) ---
0.6364 + 0.6364i
0.6364 - 0.6364i

```

Figure 5: Results of Solving the equation and getting filter coefficients

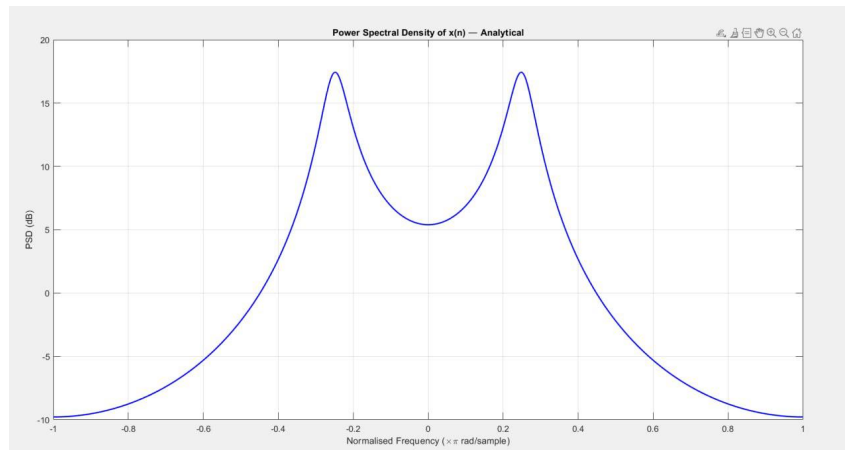


Figure 6: Power Spectral Density

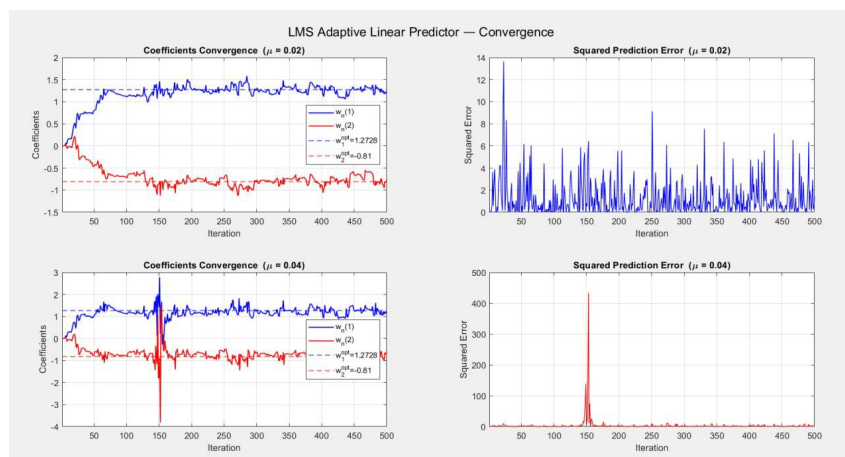


Figure 7: LMS convergence plots

We will find that at small μ we will converge slowly but at instant of convergence we will not fluctuate hardly at instant of convergence, On the other hand if we use larger μ we will converge faster but we will suffer from fluctuations around the mean.

This will clearly create a trade-off between convergence speed and stability as:

- Large μ leads to fast convergence but critical stability.
- Small μ leads to slow convergence and higher stability